

Le test take-home

6 étapes pour répondre



Structurer ta réponse au cas pratique recruteur : l'attendu, l'erreur, la ligne à écrire.

Un test take-home ne juge pas ta capacité à trouver LE bon chiffre. Il juge ta façon de raisonner face à un jeu de données inconnu, sans contrainte de temps réel. Ce que Datafuji voit dans les rendus qu'on nous partage : des candidats qui répondent à 6 questions annexes au lieu de la question posée, des notebooks avec des chemins absolus qui ne roulent plus chez le recruteur, des conclusions sans aucune reco. 6 étapes suffisent à corriger ça.

Avant de coder.

Ici, le recruteur vérifie si tu réponds à sa question, ou à celle que tu t'inventes.

1. Comprendre la question business et cadrer avant de coder

Attendu recruteur : Tu reformules l'objectif avec tes mots avant d'ouvrir un notebook, puis tu listes 2 à 3 hypothèses explicites (période, périmètre, définition d'une métrique ambiguë).

Erreur fréquente

Ouvrir directement les données et coder à l'instinct : le rendu répond alors à une question que personne n'a posée.

La ligne à écrire : en première cellule du notebook.

```
# Objectif : mesurer le taux de churn mensuel des clients actifs (>=1 achat/mois)
# Hypothese : "actif" = au moins 1 commande dans les 30 jours precedents
```

2. Explorer et valider les données avant toute analyse ÉLIMINATOIRE

Attendu recruteur : Une cellule de sanity checks avant l'analyse : volumétrie, doublons, valeurs nulles, périmètre temporel réellement couvert par le fichier.

Erreur fréquente

Sauter l'exploration et livrer un chiffre sur des données jamais vérifiées : un doublon de clé peut doubler un total sans qu'aucune erreur ne s'affiche.

La ligne à écrire : en cellule de sanity check.

```
assert df["client_id"].duplicated().any() # attendu : plusieurs commandes par client
print(df.isna().sum())
print(df["date_commande"].min(), df["date_commande"].max())
```

Pendant l'analyse.

Ici, il regarde si ton livrable se relit, et si tu réponds d'abord à ce qui a été demandé.

3. Structurer le livrable pour qu'il se relise

Attendu recruteur : Un notebook ou repo avec des sections titrées (Contexte, Exploration, Analyse, Conclusion), un chemin relatif vers les données, aucune cellule morte laissée en place.

Erreur fréquente : un fichier « brouillon.ipynb » de 40 cellules dans le désordre, avec un chemin du type C:/Users/toi/Desktop/data.csv.

```
DATA_PATH = "../data/commandes.csv" # chemin relatif, executable chez le recruteur
```

4. Répondre d'abord à LA question posée PIÈGE CLASSIQUE

Attendu recruteur : La question exacte de l'énoncé traitée en premier, avant toute extension; 1 à 2 pistes complémentaires ensuite, clairement séparées du corps de la réponse.

Erreur fréquente : partir dans 5 directions et diluer la vraie réponse au milieu d'analyses non demandées.

```
## Reponse a la question posee
## Pistes complementaires (hors perimetre initial)
```

Restituer et livrer.

Ici, il regarde si ton rendu se lance chez lui, et si ta dernière ligne est une décision.

5. Restituer un titre qui conclut et une reco actionnable

Attendu recruteur : Chaque graphique porte un titre qui donne la conclusion, pas la description des axes; la dernière cellule contient une reco en une phrase.

Erreur fréquente

Un titre du type « Ventes par mois » qui ne dit rien, suivi d'observations sans aucune décision proposée.

La ligne à écrire : titre de graphique et reco finale.

```
title="Le churn double en decembre : cibler la relance sur ce mois"
# Reco : lancer une campagne de relance mi-novembre pour les clients a risque
```

6. Rendre le rendu reproductible ÉLIMINATOIRE

Attendu recruteur : Un README court expliquant comment lancer le projet, les dépendances, la provenance des données, avec des chemins relatifs partout dans le code.

Pourquoi ce point élimine

Zéro README, un `requirements.txt` absent ou un chemin codé en dur cassent le projet dès que le recruteur clone le repo sur sa machine.

La ligne à écrire : dans le README.md.

```
## Lancer le projet
pip install -r requirements.txt
jupyter notebook analyse.ipynb
## Donnees
./data/commandes.csv (fournies dans l'enonce)
```

Ce que Datafuji vérifie en premier sur un rendu de test : est-ce qu'il tourne chez nous sans retoucher un seul chemin, et est-ce que la dernière ligne est une décision, pas une observation. Un test bien répondu ressemble à un mini rapport livré à un manager, pas à un brouillon exploratoire qu'on n'a pas eu le temps de ranger.

Fait main, en solo.

Si ce guide t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat.
Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

